COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 | Data Structures and Algorithms	**SECTION:**	CPE21S3
DATE PERFORMED:	October 06, 2026	**DATE SUBMITTED:**	October 06, 2026
NAME:	Niña Denise M. Santiago Troy Jerron A. Bautista	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

HANDS - ON ACTIVITY NO. __11.1__ **BASIC ALGORITHM ANALYSIS**

1. PROCEDURE OUTPUT

ILO A: Measure the runtime of algorithms using theoretical analysis - SANTIAGO

Code (Header File)

```
#ifndef FUNCTIONS_H
#define FUNCTIONS_H


// ================================================================
// ILO A: Measure the runtime of algorithms using theoretical analysis
// ================================================================
template <typename T>
bool search(T x, T y[], int m) {
    for (int i = 0; i < m; i++) {
        if (y[i] == x) {
            return true;
        }
    }

    return false;
}

template <typename T>
bool different(T x[], T y[], int n, int m) {
    for (int i = 0; i < n; i++) {
        if (search(x[i], y, m)) {
            return false;
        }
    }

    return true;
}
```

Code (Main File)

```cpp
#include <iostream>
#include "functions.h"

int main() {
    int x[] = {3, 8, 0, 5, 9, 2, 7};
    int y[] = {4, 9, 1, 8, 5, 2, 7};

    int n = 7;
    int m = 7;

    if (different(x, y, n, m)) {
        std::cout << "The arrays have no common values." << std::endl;
    }
    else {
        std::cout << "The arrays have common values." << std::endl;
    }

    return 0;
}
```

Output

```
The arrays have common values.
```

Code (Main File)

```cpp
#include <iostream>
#include "functions.h"

int main() {
    int x[] = {1, 2, 3, 4, 5};
    int y[] = {6, 7, 8, 9, 10};

    int n = 5;
    int m = 5;

    if (different(x, y, n, m)) {
        std::cout << "The arrays have no common values." << std::endl;
    }
    else {
        std::cout << "The arrays have common values." << std::endl;
    }

    return 0;
}
```

Output

```
The arrays have no common values.
```

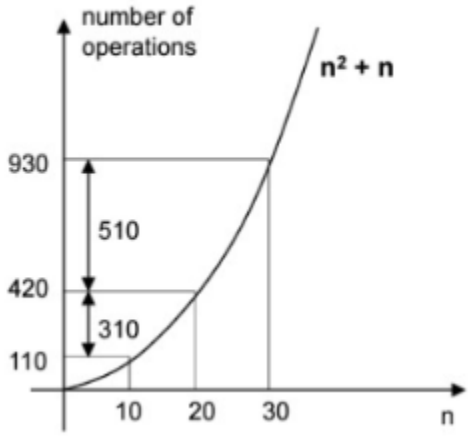
Question 1:

Given the above information, the total number of comparisons in the worst case is: ___49 Comparisons____

(Since worst case is m x n or n^2, both values in the given code is 7, so 7 x 7 is 49)

Graph Analysis

Provide an analysis of the graph for the worse case of the algorithm.



n (size of array)	number of operations
10	110
20	420
30	930

The graph shows that the number of operations increases as the size of the array also increases. The formula for the worst case is $n^2 + n$. For example, when n is 10, (10^2 + 10) there are 110 operations; when n is 20, (20^2 + 20) there are 420 operations; and when n is 30, (30^2 + 30) there are 930 operations. This shows that the number of operations grows quickly as the array becomes larger. In our case, m and n are both 7, so there are $7 \times 7 = 49$ comparisons, and the total number of operations is $7^2 + 7 = 56$. Therefore, the algorithm has a worst-case time complexity of $O(n^2)$, which means the algorithm takes more time as the size of the arrays increases.

ILO B: Analyze the results of runtime performance by comparing experimental and theoretical analysis - SANTIAGO

```cpp
#include <iostream>
#include <vector>
#include <chrono>
#include <random>
#include <algorithm>
#include <utility>

class Student {
private:
    std::pair<int, int> name;
    bool vaccinated;

public:
    Student(std::pair<int, int> n, bool v)
        : name(n), vaccinated(v) {}

    std::pair<int, int> get_name() const {
        return name;
    }

    bool is_vaccinated() const {
        return vaccinated;
    }

    bool operator==(const Student& p) const {
        return name == p.name;
    }

    bool operator<(const Student& p) const {
        return name < p.name;
    }

    bool operator>(const Student& p) const {
        return name > p.name;
    }
};
```

```cpp
// Generate a student with random information
Student generate_random_Student(int max) {
    static std::random_device rd;
    static std::mt19937 rand(rd());

    std::uniform_int_distribution<int> id_dist(1, max);
    std::uniform_int_distribution<int> vaccine_dist(0, 1);

    auto random_name = std::make_pair(
        id_dist(rand), id_dist(rand)
    );

    bool is_vaccinated = vaccine_dist(rand) == 1;

    return Student(random_name, is_vaccinated);
}

// Binary Search to determine whether vaccination is needed
bool needs_vaccination(
    const Student& p,
    const std::vector<Student>& people
) {
    auto it = std::lower_bound(
        people.begin(), people.end(), p
    );

    // Student is not found in the list
    if (it == people.end() || !(*it == p)) {
        return true;
    }

    // Student is found but has not been vaccinated
    return !it->is_vaccinated();
}

// Measure the search execution time
void search_test(int size, const Student& p) {
    std::vector<Student> people;
    people.reserve(size);

    // Generate the list of students
    for (int i = 0; i < size; i++) {
```

```
    people.push_back(generate_random_Student(size));
}

// Sort the list before searching
std::sort(people.begin(), people.end());

// Start measuring the search time
auto begin = std::chrono::steady_clock::now();

bool search_result = needs_vaccination(p, people);

// Stop measuring the search time
auto end = std::chrono::steady_clock::now();

auto duration =
    std::chrono::duration_cast<std::chrono::microseconds>(
        end - begin
    ).count();

std::cout << "Input size: " << size << '\n';
std::cout << "Time taken to search = "
          << duration << " microseconds\n";

std::cout << "Student ("
          << p.get_name().first << " "
          << p.get_name().second << ") ";

if (search_result) {
    std::cout << "needs vaccination.\n";
} else {
    std::cout << "does not need vaccination.\n";
}

std::cout << "-----------------------------------\n";
}

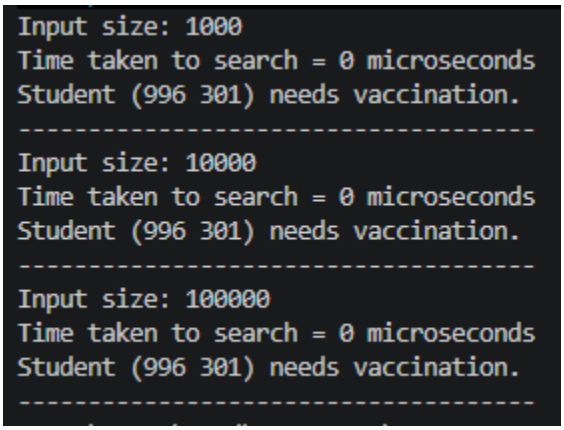
int main() {
// Generate the student to be searched
Student p = generate_random_Student(1000);

// Test the three required input sizes
search_test(1000, p);
```

```
    search_test(10000, p);
    search_test(100000, p);

    return 0;
}
```

Output



Analyses

Code Analysis

The code uses Binary Search to find a student's information in a sorted list and determine whether the student needs vaccination. It first generates a list of students with random names and vaccination statuses, then sorts the list using std::sort(). After sorting, the program searches for the target student and checks whether the student is in the list and has already been vaccinated. The std::chrono library is used to measure the search execution time in microseconds. This helps us observe how the algorithm performs with different input sizes.

Output Analysis

Based on the output, the program tested three input sizes: 1,000, 10,000, and 100,000 students. In all three tests, the search time was displayed as 0 microseconds, and the program reported that students (996, 301) needed vaccination. This happens because the search operation is very fast, and its execution time may be shorter than one whole microsecond when measured using duration_cast to microseconds. The result does not necessarily

mean that the search took absolutely no time. It shows that the measured duration was too short to register as one full microsecond. The output demonstrates that Binary Search can handle large input sizes efficiently because it repeatedly reduces the search area instead of checking every student one by one. However, since all three measurements show 0 microseconds, we cannot use these results alone to compare the exact execution times for the different input sizes.

Theoretical Analysis

The program uses Binary Search to find a student's information in a sorted list. Its time complexity is O(log n) because it divides the search area in half during each step. This means that as the number of students increases, the number of comparisons does not increase as quickly as it would with Linear Search. For the input sizes of 1,000, 10,000, and 100,000 students, Binary Search needs approximately 10, 14, and 17 comparisons in the worst case, respectively. Its space complexity is O(1) for the search operation because it uses only a fixed amount of additional memory. However, the program also stores the students in a vector, which requires O(n) space.

Experimental Analysis

Based on the actual output, the program tested 1,000, 10,000, and 100,000 students. All three tests showed an execution time of 0 microseconds, and the program reported that students (996, 301) needed vaccination. This means that the measured search operation was very fast, but the timer did not record a duration of at least one whole microsecond. The result does not mean that the search took no time at all. Although the program successfully produced results for all three input sizes, the recorded times are not enough to compare the exact speed of each test. Measuring the search repeatedly and calculating the average execution time could provide more useful results.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO A: Measure the runtime of algorithms using theoretical analysis & ILO B: Analyze the results of runtime performance by comparing experimental and theoretical analysis

Problem 1: Consider a program that returns true if the elements in an array are unique

Code - SANTIAGO

```cpp
#include <iostream>
#include <vector>
#include <chrono>
#include <random>
#include <iomanip>

// Returns false if duplicate elements are found
bool Unique(const std::vector<int>& A) {
    int n = static_cast<int>(A.size());

    for (int i = 0; i < n; i++) {
        for (int j = i + 1; j < n; j++) {
            if (A[i] == A[j]) {
                return false;
            }
        }
    }

    return true;
}

int main() {
    const int sizes[] = {100, 500, 1000, 2000};
    std::mt19937 generator(42);

    std::cout << std::fixed << std::setprecision(3);
    std::cout << "PROBLEM\n\n";

    for (int size : sizes) {
        std::vector<int> A(size);

        // Generate unique values
        for (int i = 0; i < size; i++) {
```

```
        A[i] = i;
    }

    // Shuffle the values
    for (int i = size - 1; i > 0; i--) {
        std::uniform_int_distribution<int> dist(0, i);
        int j = dist(generator);
        int temp = A[i];
        A[i] = A[j];
        A[j] = temp;
    }

    auto start = std::chrono::steady_clock::now();

    bool result = Unique(A);

    auto end = std::chrono::steady_clock::now();

    double time =
        std::chrono::duration<double, std::micro>(
            end - start
        ).count();

    std::cout << "Input size: " << size << '\n';
    std::cout << "All elements unique: "
              << (result ? "Yes" : "No") << '\n';
    std::cout << "Execution time: "
              << time << " microseconds\n";
    std::cout << "---------------------------------------\n";
  }

  return 0;
}
```

Code Output - SANTIAGO

```
PROBLEM

Input size: 100
All elements unique: Yes
Execution time: 53.600 microseconds
---------------------------------------
Input size: 500
All elements unique: Yes
Execution time: 785.800 microseconds
---------------------------------------
Input size: 1000
All elements unique: Yes
Execution time: 3063.700 microseconds
---------------------------------------
Input size: 2000
All elements unique: Yes
Execution time: 12757.100 microseconds
---------------------------------------
```

Analyses

Theoretical Analysis

The Unique(A) algorithm checks whether an array contains duplicate elements by comparing each element with the elements that come after it. It uses two nested loops, which means that more comparisons are needed as the input size increases. In the worst case, when all elements are unique, the algorithm performs n(n-1)/2 comparisons. Therefore, its time complexity is O(n^2), while its space complexity is O(1) because it uses only a fixed amount of extra memory.

Experimental Analysis

Based on the experimental results, the program tested arrays with 100, 500, 1,000, and 2,000 elements. The execution times were 57.100, 779.400, 3,230.300, and 12,389.500 microseconds, respectively. All tests displayed “All elements unique: Yes,” which means that no duplicate elements were found in the arrays. As the input size increased, the execution time also increased because the algorithm needed to compare more pairs of elements. For example, when the input size increased from 1,000 to 2,000 elements, the execution time increased from

3,230.300 to 12,389.500 microseconds, which is almost four times longer. These results support the theoretical analysis that the algorithm has a time complexity of O(n^2). This shows that the algorithm becomes slower as the input size grows because it performs many comparisons to check whether all elements are unique.

Analysis and Comparison

The theoretical analysis shows that the algorithm has a time complexity of O(n^2), and the experimental analysis allows us to observe how its execution time changes with different input sizes. For 100 elements, the algorithm performs 4,950 comparisons. For 500 elements, it performs 124,750 comparisons; for 1,000 elements, it performs 499,500 comparisons; and for 2,000 elements, it performs 1,999,000 comparisons. These values show that doubling the input size from 1,000 to 2,000 results in approximately four times as many comparisons. The actual execution time may not increase by exactly four times, but the results should generally follow the same pattern. This shows that the algorithm can work well for small arrays but becomes less efficient as the input size grows.

Problem 2: Consider another program that raises a number x to an arbitrary nonnegative integer, n.

Code - SANTIAGO

```
#include <iostream>
#include <chrono>
#include <iomanip>

// Regular recursive power: O(n)
long long rpower(int x, int n) {
    if (n == 0) {
        return 1;
    }

    return x * rpower(x, n - 1);
}

// Binary recursive power: O(log n)
long long brpower(int x, int n) {
    if (n == 0) {
        return 1;
    }

    if (n % 2 != 0) {
        long long y = brpower(x, (n - 1) / 2);
        return x * y * y;
    }

    long long y = brpower(x, n / 2);
    return y * y;
}

int main() {
    const int base = 2;
    const int exponents[] = {5, 10, 15, 20};

    std::cout << std::fixed << std::setprecision(3);
    std::cout << "PROBLEM 2\n\n";

    for (int n : exponents) {
        auto start1 = std::chrono::steady_clock::now();
```

```
    volatile long long result1 = rpower(base, n);
    auto end1 = std::chrono::steady_clock::now();

    double time1 =
        std::chrono::duration<double, std::micro>(
            end1 - start1
        ).count();

    auto start2 = std::chrono::steady_clock::now();
    volatile long long result2 = brpower(base, n);
    auto end2 = std::chrono::steady_clock::now();

    double time2 =
        std::chrono::duration<double, std::micro>(
            end2 - start2
        ).count();

    std::cout << "Base: " << base
              << ", Exponent: " << n << '\n';

    std::cout << "rpower result: " << result1 << '\n';
    std::cout << "rpower time: "
              << time1 << " microseconds\n";

    std::cout << "brpower result: " << result2 << '\n';
    std::cout << "brpower time: "
              << time2 << " microseconds\n";

    std::cout << "--------------------------------\n";
  }

  return 0;
}
```

Code Output - SANTIAGO

```
PROBLEM 2

Base: 2, Exponent: 5
rpower result: 32
rpower time: 0.100 microseconds
brpower result: 32
brpower time: 0.100 microseconds
---------------------------------
Base: 2, Exponent: 10
rpower result: 1024
rpower time: 0.100 microseconds
brpower result: 1024
brpower time: 0.100 microseconds
---------------------------------
Base: 2, Exponent: 15
rpower result: 32768
rpower time: 0.100 microseconds
brpower result: 32768
brpower time: 0.000 microseconds
---------------------------------
Base: 2, Exponent: 20
rpower result: 1048576
rpower time: 0.100 microseconds
brpower result: 1048576
brpower time: 0.100 microseconds
---------------------------------
```

Analyses

Theoretical Analysis

The code presents two algorithms, rpower(x, n) and brpower(x, n), which calculate the power of a number using recursion. The rpower algorithm decreases the exponent by one in every recursive call, giving it a time complexity of O(n) and a space complexity of O(n). Meanwhile, brpower divides the exponent by two in each recursive call, giving it a time complexity of O(log n) and a space complexity of O(log n). This means that brpower requires fewer recursive calls as the exponent increases, making it theoretically more efficient for large exponents.

Experimental Analysis

Based on the experimental results, both rpower and brpower were tested using base 2 and exponents 5, 10, 15,

and 20. Both algorithms produced the same correct results: 32, 1,024, 32,768, and 1,048,576, respectively. The execution time of rpower was consistently 0.100 microseconds for all four tests, while brpower recorded 0.100, 0.100, 0.000, and 0.100 microseconds. These results show that both algorithms completed the calculations very quickly, making their performance difficult to compare using these measurements alone. The 0.000 microseconds recorded for brpower at exponent 15 does not mean that the calculation took absolutely no time; it means the measured duration was too short to register as a full microsecond. Although the experimental results do not clearly show which algorithm is faster, the theoretical analysis indicates that brpower is more efficient for larger exponents because it reduces the exponent by half during each recursive call, while rpower decreases it by one.

Analysis and Comparison

The theoretical and experimental analyses help explain why two algorithms that solve the same problem can have different levels of efficiency. The rpower algorithm has a time complexity of O(n), while brpower has a time complexity of O(log n). For example, with an exponent of 20, rpower makes 21 recursive calls, while brpower makes approximately 6 recursive calls. This shows that brpower can reduce the number of calculations needed to obtain the same result. The experimental results may not always show a large difference because the tested exponents are small and the execution times are short. Nevertheless, the theoretical analysis shows that brpower is a better choice for larger exponents because its number of recursive calls grows much more slowly.

Analysis and Comparison of Problem 1 and Problem 2

Comparing the experimental results, Problem 1 showed a clear increase in execution time as the input size grew. The time increased from 57.100 microseconds for 100 elements to 779.400 microseconds for 500 elements, 3,230.300 microseconds for 1,000 elements, and 12,389.500 microseconds for 2,000 elements. This shows that the Unique algorithm became much slower as it checked more elements. In Problem 2, both rpower and brpower produced the same correct answers for all four exponents. However, their execution times were almost identical: both recorded 0.100 microseconds in most tests, except brpower, which recorded 0.000 microseconds at

exponent 15. Unlike Problem 1, the results in Problem 2 did not show a clear increase in execution time or a consistent speed difference between the two algorithms. This is because the calculations were very fast and the measured times were too small to show their performance differences clearly. Overall, Problem 1 provided a clearer example of how execution time increases with input size, while Problem 2 showed that theoretical analysis is important when experimental results are too close to determine which algorithm is more efficient.

ILO A: Measure the runtime of algorithms using theoretical analysis & ILO B: Analyze the results of runtime performance by comparing experimental and theoretical analysis

Problem 1: Consider a program that returns true if the elements in an array are unique

Code (Header File) - BAUTISTA

```
#ifndef SUPPLEMENTARY_H
#define SUPPLEMENTARY_H

// Problem 1: Consider a program that returns true if the elements in an array are unique.
bool unique(int A[], int n) {
    for (int i = 0; i <= n - 1; i++) {
        for (int j = i + 1; j <= n - 1; j++) {
            if (A[i] == A[j]) {
                return false;
            }
        }
    }
    return true;
}
```

Code (Main File) - BAUTISTA

```
#include <iostream>
#include <chrono>
#include "supplementary.h"

const int size = 10;

int main() {
    // For experimental analysis
    auto start = std::chrono::steady_clock::now();
    auto end = std::chrono::steady_clock::now();
    auto time = std::chrono::duration_cast<
        std::chrono::microseconds
        >(end - start).count();

    // Problem 1:
```

Output - BAUTISTA

```
The numbers inside the array are unique.
Time taken: 0 microseconds
```

Theoretical Analysis - BAUTISTA

In this program, I observed that the time complexity of this code is $\frac{n^2-n}{2}$ because it compares elements with the elements that come after it. For this program, n = 10, and using the formula, T(n) = 45 because in the first iteration of the loop, it compares the elements in the array 9 times, and the comparison decrements in the next iteration until it finds a duplicate number or all the numbers in the array have been compared.

Experimental Analysis - BAUTISTA

In this program, its space complexity is S(n) = 4n + 8 because, in the function unique, the array size n takes up 4 bytes, and combined with n to get 4n, then both the iterators i and j are 4 bytes each, so 8 bytes in total; and with array size n = 10, the space complexity of the program is 48 bytes.

Problem 2: Consider another program that raises a number x to an arbitrary nonnegative integer, n

Code (Header File) - BAUTISTA

```
// Problem 2: Consider another program that raises a number x to an arbitrary nonnegative integer, n
int rpower (int x, int n) {
    if (n == 0) return 1;
    else return x*rpower(x, n-1);
}

int brpower(int x, int n) {
    if (n == 0) return 1;
    if (n % 2 == 1) {
        int y = brpower(x, (n-1)/2);
        return x*y*y;
    }
    else if (n % 2 == 0) {
        int y = brpower(x, n/2);
        return y*y;
    }
}
```

Code (Main File) - BAUTISTA

```
// Problem 2:
start;
int rproduct = rpower(2, 4);
end;
std::cout <<"\nAnswer using rpower function: " <<rproduct <<std::endl;
time;
std::cout << "Time taken: " << time << " microseconds\n";

start;
int brproduct = brpower(2, 4);
end;
std::cout <<"\nAnswer using brpower function: " <<brproduct <<std::endl;
time;
std::cout << "Time taken: " << time << " microseconds\n";
```

Output - BAUTISTA

```
Answer using rpower function: 16
Time taken: 0 microseconds

Answer using brpower function: 16
Time taken: 0 microseconds
```

Conclusion / Analysis / Learnings
In this module, I learned how to study algorithms both in theory and in practice. I understand that theoretical analysis helps us predict how an algorithm grows as the input size increases, while experimental analysis shows us the actual time it takes when running the program. For example, I saw that simple search and comparison algorithms can have a time complexity of O(n^2), which means they become slower very quickly as the input size grows. On the other hand, binary search has a time complexity of O(logn), which makes it much more efficient for large inputs. I also learned that sometimes experimental results may show very small execution times that do not clearly reflect the differences between algorithms, so theoretical analysis is important to explain why one algorithm is better than another. Overall, this activity taught me that analyzing both runtime and space complexity is necessary to understand the efficiency of algorithms, and that good algorithms should be designed to handle larger inputs effectively.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: ***Grammarly and Microsoft Copilot***

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: ***Grammar correction*** **and Explanation of Concepts**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **I used Microsoft Copilot to better understand the codes, and Grammarly to refine grammar and vocabulary. These tools supported my work but did not replace my own thinking, analysis, authorship, or completion of the activity.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.



I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.